Trigger et Procedures stockées avec Mysql

Qu'est-ce qu'un trigger ?

Tout comme les procédures stockées, les triggers servent à exécuter une ou plusieurs instructions. Mais à la différence des procédures, il n'est pas possible d'appeler un trigger : un trigger doit être déclenché par un événement dit événement moteur.

Un trigger est **attaché à une table**, et peut **être déclenché** par :

- une insertion dans la table (requête INSERT) ;
- la suppression d'une partie des données de la table (requête DELETE) ;
- la modification d'une partie des données de la table (requête UPDATE).

Par ailleurs, une fois le trigger déclenché, ses instructions peuvent être exécutées soit juste avant l'exécution de l'événement déclencheur, soit juste après.

Que fait un trigger ?

Un trigger exécute un **traitement pour chaque ligne insérée, modifiée ou supprimée** par l'événement déclencheur. Donc si l'on insère cinq lignes, les instructions du trigger seront exécutées cinq fois, chaque itération permettant de traiter les données d'une des lignes insérées.

À quoi sert un trigger ?

On peut faire de nombreuses choses avec un trigger .Les usages fréquents de ces sont les suivants :

Respect de contraintes et vérifications de données
Intégrité des données
Historisation des actions
Mise à jour d'informations qui dépendent d'autres données

Limites ?

Il est impossible de travailler avec des transactions à l'intérieur d'un trigger.
(Pas de `START TRANSACTION`, `COMMIT` ou `ROLLBACK`**)**
Cette restriction est nécessaire, puisque la requête ayant provoqué l'exécution du trigger pourrait très bien se trouver elle-même à l'intérieur d'une transaction. Auquel cas, toute commande `START TRANSACTION`, `COMMIT` ou `ROLLBACK` interagirait avec cette transaction, de manière intempestive.

Les requêtes préparées ne peuvent pas non plus être utilisées.

Syntaxe d'un trigger ?

```
CREATE TRIGGER nom_trigger moment_trigger evenement_trigger
ON nom_table FOR EACH ROW
contenu_trigger;
```

exemple :

```
CREATE TRIGGER after_insert_personne AFTER INSERT
ON Personne FOR EACH ROW
BEGIN
IF NEW.sexe IS NOT NULL   -- le sexe n'est ni NULL
   AND NEW.sexe != 'M'       -- ni "M"
   AND NEW.sexe != 'F'       -- ni "F"
     THEN
```

```
      SET NEW.sexe = NULL;
   END IF;
END;
```

La commande « delimiter » permet au client MySQL en ligne de commande de ne pas couper les lignes sur les « ; » qui sont dans le code des triggers. Si les commandes sont collées dans un phpMyAdmin, il faut l'inclure.
On écrira donc plutôt

```
DELIMITER |
CREATE TRIGGER after_insert_personne AFTER INSERT
ON Personne FOR EACH ROW
BEGIN
//Actions
END|
DELIMITER ;
```

Adaptons le trigger précédent de façon à insérer l'erreur constatée dans la table erreurs :

```
DELIMITER |
CREATE TRIGGER before_insert_animal BEFORE INSERT
ON Animal FOR EACH ROW
BEGIN
   IF NEW.sexe IS NOT NULL   -- le sexe n'est ni NULL
   AND NEW.sexe != 'M'       -- ni "M"
   AND NEW.sexe != 'F'       -- ni "F"
     THEN
       INSERT INTO Erreur (erreur) VALUES ('Erreur : sexe doit valoir "M", "F" ou NULL.');
   END IF;
END |
DELIMITER ;
```

Procédures stockées

Les procédures stockées sont disponibles depuis la version 5 de MySQL, et permettent d'automatiser des actions, qui peuvent être très complexes.
Une procédure stockée est en fait une **série d'instructions SQL** désignée par un **nom**. Lorsque l'on crée une procédure stockée, on l'enregistre **dans la base de données** que l'on utilise, au même titre qu'une table par exemple. Une fois la procédure créée, il est possible d'**appeler** celle-ci, par son nom. Les instructions de la procédure sont alors exécutées.

Syntaxe d'une procédure stockée ?

```
CREATE PROCEDURE afficher_gammes ()
BEGIN
   SELECT id, libelle, gamme_id, prix FROM Gamme;
END;
```

Le principe est bon mais MySQL va le refuser pour un problème de séparateur.
Le problème puisque c'est le premier **;** rencontré par l'instruction `CREATE PROCEDURE`, qui naturellement pense devoir s'arrêter là. Ceci déclenche une erreur puisqu'en réalité, l'instruction `CREATE PROCEDURE` n'est pas terminée : le bloc d'instructions n'est pas complet !

Changeons donc de délimiteur :

```
DELIMITER | -- On change le délimiteur
CREATE PROCEDURE afficher_gammes ()
BEGIN

SELECT id, libelle, gamme_id, prix FROM Gamme;
-- Cette fois, le ; ne nous embêtera pas
END|
```

Utilisation d'une procédure stockée

Pour appeler une procédure stockée, il faut utiliser le mot-clé `CALL`, suivi du nom de la procédure appelée, puis de parenthèses (avec éventuellement des paramètres).

```
CALL afficher_gammes ()
```

Voyons la procédure stockée suivante : compter_materiels_selon_gamme définie avec deux paramètres : un entrant et un sortant :

```
DELIMITER |
CREATE PROCEDURE compter_materiels_selon_gamme (IN p_gamme_id INT, OUT
p_nb_materiels INT)
BEGIN
   SELECT COUNT(*) INTO p_nb_materiels
   FROM Materiels
   WHERE gamme_id = p_gamme_id;
END |
DELIMITER ;
```

Et maintenant ?

Il faut appeler la procédure stockée :

```
CALL compter_materiels_selon_gamme (2, @nb_materiels_gamme);
```

et faire quelque chose du résultat :

```
SELECT @nb_materiels_gamme;
```

Supprimer une procédure stockée ?

DROP PROCEDURE compter_materiels_selon_gamme**;**

Ce qu'il faut retenir ?

Une procédure stockée est un objet de la base de données **stocké de manière durable**, **comme une table.**

Les procédures stockées peuvent permettre de **gagner en performance** en diminuant les allers-retours entre le client et le serveur. Elles peuvent

également aider à **sécuriser une base de données** et à s'assurer que les traitements sensibles soient toujours exécutés de la même manière. En revanche, elle **ajoute à la charge du serveur** et sa syntaxe n'est **pas toujours portable** d'un SGBD à un autre.

On les utilise comment ?

• *Tout d'abord, une procédure peut aussi être…une fonction stockée*

Par exemple :

```
DELIMITER //
DROP FUNCTION IF EXISTS compter //
CREATE FUNCTION compter () RETURNS INT
BEGIN
   SELECT count(*) FROM user INTO @result;      RETURN @result;
END//
DELIMITER ;
```

SELECT compter ();//Retourne le nombre d'entrées dans la table user.

• *Voyons cette fonction plus utile*

La fonction login_existe renvoie 0 ou 1 si le login reçu en paramètre est déjà dans la table

```
DELIMITER //
DROP FUNCTION IF EXISTS login_existe //
CREATE FUNCTION login_existe (newlogin VARCHAR(255)) RETURNS INT
BEGIN
   SELECT count(*) FROM user WHERE login = newlogin INTO @result;
   RETURN @result;
END//
DELIMITER ;
SELECT login_existe('jvallade');
```

Et combinons là avec un trigger qui annule l'insertion si le login est déjà présent dans la table user :

```
DELIMITER //
DROP TRIGGER IF EXISTS valide_insertion //
CREATE TRIGGER valide_insertion BEFORE INSERT ON user FOR EACH ROW
BEGIN
   IF login_existe(NEW.login) THEN         SET NEW.login=NULL;      END IF;
END //
DELIMITER ;
```

Joli non ?